

PART - 3



Angular Interview Questions - Theory & Conceptual Explanations



Q21: What are Observables?

Explanation :

Observables are lazy collections of multiple values over time. Part of Reactive Extensions (RxJS).

Key Concepts:

- **Observable:** Data stream
- **Observer:** Consumer of data stream
- **Subscription:** Execution of observable
- **Operators:** Functions to transform streams

Observable vs Promise:

- **Observable:** Multiple values, cancellable, lazy, operators
- **Promise:** Single value, eager, not cancellable



Q22: Common RxJS Operators

Explanation :

- **Creation:** of, from, interval, timer
- **Transformation:** map, switchMap, mergeMap, concatMap
- **Filtering:** filter, take, skip, debounceTime, distinctUntilChanged
- **Combination:** combineLatest, forkJoin, merge, concat
- **Error Handling:** catchError, retry, retryWhen
- **Utility:** tap, delay, timeout



Q23: Subjects vs Observables

Explanation :

- **Observable:**
 - Unicast (each subscriber gets own execution)
 - Read-only
 - Cold by default
- **Subject:**
 - Multicast (same execution shared)
 - Can emit values (.next())
 - Hot by default
- **Types of Subjects:**
 - **Subject:** No initial value, no replay
 - **BehaviorSubject:** Has initial value, replays current value
 - **ReplaySubject:** Replays specified number of values
 - **AsyncSubject:** Emits only last value on completion



Q24: State Management Approaches

Explanation :

1. Service with BehaviorSubject:

- Simple applications
- Local component state
- Shared observable streams

2. NgRx (Redux pattern):

- Large applications
- Complex state
- Predictable state changes
- Time-travel debugging

3. Akita:

- Simpler than NgRx
- Less boilerplate
- Built on RxJS

4. NGXS:

- Similar to NgRx
- Less boilerplate
- Uses TypeScript decorators



Q25: What is NgRx?

Explanation :

NgRx implements Redux pattern in Angular:

Core Concepts:

- Store: Single source of truth
- Actions: Describe events
- Reducers: Pure functions that handle state changes
- Selectors: Extract pieces of state
- Effects: Handle side effects

Principles:

- Single source of truth
- State is read-only
- Changes via pure functions
- Unidirectional data flow



Q26: How Change Detection Works

Explanation :

Angular checks for changes and updates DOM accordingly.

Change Detection Strategies:

1. **Default:** Checks all components on every event

2. **OnPush:** Checks only when:

- Input reference changes
- Event emitted from component
- Async pipe receives new value
- Manually triggered

Optimization Techniques:

- Use OnPush strategy
- Use pure pipes
- Use trackBy in *ngFor
- Minimize expressions in templates
- Use immutable data structures



Q27: Zone.js and Change Detection

Explanation :

Zone.js monkey-patches async operations to trigger change detection.

How it works:

- Wraps async operations
- Notifies Angular when async operations complete
- Triggers change detection cycle

When to bypass:

- High-frequency events
- Animation frames
- Using `runOutsideAngular()`



Q28: Performance Optimization Techniques

Explanation :

1. Bundle Optimization:

- Lazy loading
- Tree shaking
- Code splitting
- AOT compilation

2. Runtime Optimization:

- OnPush change detection
- Pure pipes
- trackBy in *ngFor
- Virtual scrolling

3. Memory Optimization:

- Unsubscribe from observables
- Use WeakMap/WeakSet
- Clean up event listeners
- Use OnPush strategy



Q29: AOT vs JIT Compilation

Explanation :

JIT (Just-in-Time):

- Compiles in browser at runtime
- Larger bundle size
- Development mode

AOT (Ahead-of-Time):

- Compiles during build
- Smaller bundle size
- Faster rendering
- Template errors caught early
- More secure
-



Q30: Security Best Practices

Explanation :

1. XSS Prevention:

- Automatic sanitization
- Use DomSanitizer for trusted content
- Avoid innerHTML with user data

2. CSRF Protection:

- Use HttpClient with built-in protection
- Implement anti-CSRF tokens

3. Content Security Policy (CSP):

- Configure HTTP headers
- Restrict resource loading

4. Authentication/Authorization:

- Route guards
- JWT tokens
- Secure HTTP headers